FOR COMPLETE BEGINNERS

Bread first? Not here.

Complete Modern Java, episode 23: looping through a map, in Java and TypeScript

> Three ways through a map: only the keys, only the values, or each key with its value.

Bread, Milk, Eggs and Rice went into a HashMap in that order. Printing every name gave Eggs, Bread, Milk, Rice.

A HashMap does not promise to keep the order you added things in. Yours may differ; both are correct.

In this episode

- keySet()
- A key, then get
- values()
- entrySet() and Map.Entry
- Order is the map's own
- TypeScript Map loop

Coming next

- static and constants (episode 24)

At the time of recording: Java 25, Node 24 LTS, IntelliJ IDEA 2026.1.3.

01 What each run printed

keySet loop	`Eggs, Bread, Milk, Rice`
keySet loop with get(name)	`Eggs: 250, Bread: 150, Milk: 99, Rice: 199`
values loop, adding the prices	`698`
entrySet loop	`Eggs costs 250 ... Rice costs 199`
menu.ts: for (const entry of catalog)	`Bread: 150, Milk: 99, Eggs: 250, Rice: 199`

Every row was run, not predicted. The Java order is what this run produced, not a promise.

02 Only the names: keySet()

keySet() gives the keys, and the for-each from episode 15 walks through them. Inside the loop, get(name) finds each product.

It works, but it asks the map again for every name.

Menu.java

```
for (String name : catalog.keySet()) {
    Product product = catalog.get(name);
    IO.println(name + ": " + product.getPriceCents());
}
```

> Printed Eggs: 250, Bread: 150, Milk: 99, Rice: 199.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

03 Only the products: values()

When you only need the values, loop over values(). Adding one of each product gives 698.

Menu.java

```
int totalCents = 0;
for (Product product : catalog.values()) {
    totalCents += product.getPriceCents();
}
IO.println(totalCents);   // 698
```

> Printed 698.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

04 Both at once: entrySet()

Map.Entry is the name Java gives one key with its value. getKey and getValue, without calling get again.

Menu.java

```
for (Map.Entry<String, Product> entry : catalog.entrySet()) {
    IO.println(entry.getKey() + " costs " + entry.getValue().getPriceCents());
}
```

> Printed Eggs costs 250, Bread costs 150, Milk costs 99, Rice costs 199. No import was needed in the compact file.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

05 TypeScript Map

Each entry is a key and a value: entry[0] and entry[1]. A TypeScript Map keeps the order you set things in.

menu.ts

```
for (const entry of catalog) {
    const name = entry[0];
    const product = entry[1];
    console.log(name + ": " + product.priceCents);
}
```

> tsc was quiet; node printed Bread: 150, Milk: 99, Eggs: 250, Rice: 199.
>
> Run on a Mac, 2026-11-07: Java 25.0.4, Node 24.22.0, TypeScript 7.0.2.

Loop through a map

1. Only the names? keySet().
2. Only the values? values().
3. Both? entrySet(), with getKey and getValue.
4. Never count on a HashMap's order. Need it? LinkedHashMap, later.
5. Challenge: add Sugar at 180. The values total? 878.

Everything else

Site
https://code-with-sam-dev.github.io

YouTube
https://www.youtube.com/@CodewithSam-Dev

LinkedIn
https://www.linkedin.com/company/code-with-sam-dev

X
https://x.com/CodeWithSamDev

TikTok
https://www.tiktok.com/@codewithsamdev

Instagram
https://www.instagram.com/codewithsamdev

Facebook
https://www.facebook.com/codewithsamdev

GitHub
https://github.com/code-with-sam-dev

Buy me a coffee
https://buymeacoffee.com/codewithsam

Buy me sadza
https://buymesadza.com/codewithsam

The code and the challenge files: complete-modern-java
https://github.com/code-with-sam-dev/complete-modern-java

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Java is a trademark of Oracle. Node.js is a trademark of the OpenJS Foundation. TypeScript is a trademark of Microsoft. IntelliJ IDEA is a trademark of JetBrains. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them.

Technical claims verified 2026-11-07 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.